Introducción a los algoritmos genéticos

Ejercicio

Se desea encontrar el máximo de la función

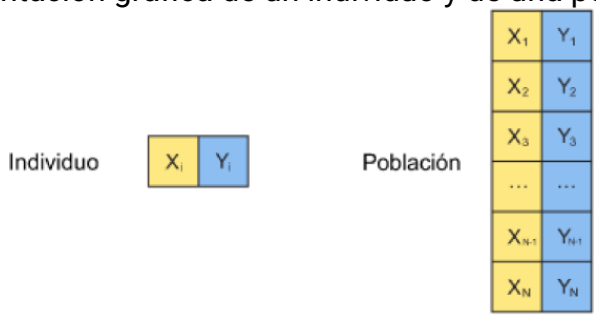
$$f(x,y) = \sqrt{x^2 + y^2}$$

En el dominio

$$\{x,y\} \in [-100,100]$$

En este simple ejemplo, las variables independientes son x e y, y la **función objetivo** es $f(x,y)$.

La *imagen* muestra la representación gráfica de un individuo y de una población de n individuos.



Para una población de n= 10, se muestran soluciones aleatorias iniciales:

Individuo	x	y
1	68.88	81.62
2	51.59	0.93
3	-15.88	-43.63
4	-48.21	51.16
5	2.25	23.67
6	-19.01	-49.89
7	56.75	81.94
8	-39.33	96.55
9	-4.68	62.04
10	16.67	80.43

Soluciones óptimas del ejemplo

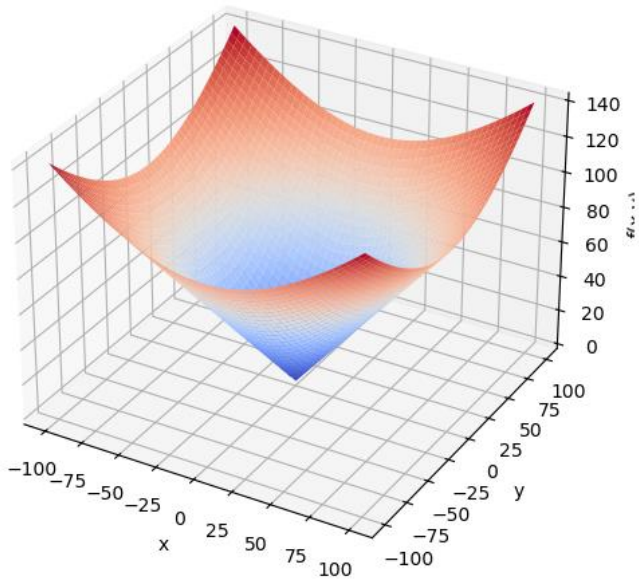
Individuo	x	y
1	100	100
2	-100	100
3	-100	-100
4	100	-100

El problema tiene *cuatro posibles soluciones óptimas*.

Representación gráfica de la función de optimización

```
from matplotlib import cm

# Definimos la función objetivo
def funcion_prueba(x):
    res = np.sqrt (x[0] ** 2 + x[1] ** 2)
    return res

# Generamos vectores que barran todo el dominio
x = np.linspace (-100, 100, 1000)
y = np.linspace (-100, 100, 1000)

# Evaluamos todos los valores de las variables
x_ax, y_ax = np.meshgrid (x, y)
vals = np.c_ [x_ax.ravel () , y_ax.ravel ()]
fx = np.reshape ([funcion_prueba(val) for val in vals], (1000, 1000))

# Representamos los resultados obtenidos
figure_3d = plt.figure(figsize=(8,6))
ax = figure_3d.add_subplot(111, projection="3d")

ax.plot_surface (x_ax, y_ax, fx, cmap=cm.coolwarm)
ax.set_xlabel ("x")
ax.set_ylabel ("y")
ax.set_zlabel ("f(x,y)")
```

```
Text(0.5, 0, 'f(x,y)')
```



La gráfica obtenida muestra el *espacio de búsqueda*, que es el conjunto de valores que pueden tomar las variables independientes, es decir, el dominio de la función.

Las cuatro **soluciones óptimas** del problema corresponden a los **cuatro picos** de la superficie.

Codificación del algoritmo genético

1. Instalación del módulo *deap*

```
pip install deap
```

2. Librerías necesarias

```
import random
import math
import numpy as np
from deap import base
from deap import creator
from deap import tools
from deap import algorithms
import matplotlib.pyplot as plt
```

3. Definición del problema
    - Tipo de problema de optimización: **maximización**
    - Tipo de objeto: **Lista**
    - Toolbox:
        - Función para crear individuo
        - Función para crear la población
4. Creación del problema

   Se utiliza el método `create()` de la clase `creator`, para crear la clase llamada `FitnessMax`, subclase de la clase base `base.Fitness`, agregando el atributo `weights`.

```
create (<name>,  <base>, <attribute>)
```

```
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
```

   `weights` contendrá una tupla con tantas componentes como objetivos tenga el problema, y con un valor que indicará si estos objetivos son de **maximización** o minimización.
    - Si el problema es de maximización `weights` tiene el valor de `(1.0, )`
    - Si el problema es de minimización `weights` tiene el valor de `(-1.0, )`

5. Creación del individuo
   El método `créate()` se utiliza para definir la clase que encapsula al individuo:

```
creator.create("Individual", list, fitness=creator.FitnessMax)
```

   En esta línea de código, estamos creando una clase que se denomina `Individual`, que hereda de la clase `list` (por lo tanto, tiene todos los métodos de una lista) y que contiene el atributo `fitness`, el cual ha sido inicializado con el objeto `FitnessMax` creado en la anterior línea. Es decir, el individuo será una lista que tiene un atributo `fitness` que almacenará la calidad o desempeño de este.

    - Se definió el tipo de problema y el tipo de individuo a utilizar.
    - Estos <u>dos pasos se van a dar siempre</u> y, en la mayoría de los casos, ambas líneas de código se repetirán con pequeñas modificaciones (dependiendo del número de objetivos y del tipo de objeto que almacene los individuos).
    - Definir los individuos como una lista de variables es un procedimiento muy eficiente y flexible, ya que cada variable independiente será una posición de la lista.
    - El tamaño de la lista se define cuando se crean los individuos de la población inicial, como veremos más adelante.

6. Crear individuos aleatorios y población inicial

   Se deben definir funciones que permitan *crear individuos aleatorios* que establezcan la *población inicial*.

```
toolbox = base.Toolbox()
```

   Después, se deben registrar funciones que permitan crear individuos aleatorios (gen del cromosoma). El método `register()` registra una función en el objeto `toolbox` con el nombre `attr_uniform`. Este nombre es un alias que se puede definir a criterio del programador.

```
# Generación de genes
toolbox.register("attr_uniform", random.uniform, -100, 100)
```

   En este caso particular, se llama a la función `random.uniform()` con los parámetros que se indican a continuación `(-100,100)`

   A continuación, y con el fin de crear el individuo completo, se requiere llamar a la función que genera cada uno de los genes tantas veces como variables independientes tengamos. Para ello, se registra una función que se denomina `individual`. A su vez, esta función llama a la función `tools.initRepeat` de la siguiente forma:

```
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_uniform, 2)
```

   Así, cada vez que se ejecute `toolbox.inidivual()` se creará un individuo aleatorio. Es importante recordar que individuo es una lista que tiene un atributo fitness donde se almacena la calidad del mismo.

   Ahora se creará la **población inicial**

```
toolbox.register("population", tools.initRepeat, list, toolbox.individual, 10)
```

   En esta sentencia la función que se registra se llama `population`, la cual utiliza `initRepeat` para llamar diez veces a la función individual (se llama una vez por cada individuo que formará la población inicial). El resultado se guarda en una lista que contiene la población inicial generada. Es decir, con respecto a los argumentos de `initRepeat`, el *container* es una lista, la función *func* es `toolbox.individual` y `n = 10`

7. Función objetivo.

   Se definirá la función objetivo como una función de Python.

```python
def funcion_objetivo(x):
    for i in range(len(x)):
        if x[i] > 100 or x[i] < -100:
            return -1,
    res = math.sqrt(x[0]**2 + x[1]**2)
    return res,
```

Aplicación del fitness

```python
toolbox.register("evaluate", funcion_objetivo)
```

La función *fitness* devuelve una tupla, con independencia del número de objetivos del problema. Por lo tanto, no hay que olvidar que la función objetivo siempre debe devolver una tupla, aunque una de las componentes esté vacía.

8. Operadores genéticos
9.
    - Selección: `select`, Cruza: `mate`, Mutación: `mutate`

En esta librería a **selección** es un procedimiento siempre *elitista*, de forma que un individuo tendrá mayor probabilidad de dejar descendencia si su fitness es más adecuado al problema de optimización.

La operación de **cruza** es una operación *probabilística* que permite que dos individuos seleccionados crucen o intercambien su información genética para crear dos nuevos individuos.

La operación de **mutación** es una operación *probabilística* que permite que un individuo seleccionado modifique su información genética para crear un nuevo individuo.

```python
toolbox.register("mate", tools.cxOnePoint)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma= 5, indpb=0.1)
toolbox.register("select", tools.selTournament, tournsize=3)
```

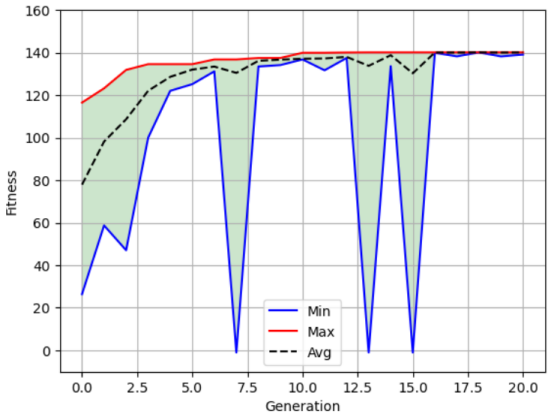
10. Configuración del algoritmo genético

```python
def main():
    random.seed(42) #Semilla del generador de los número pseudoaleatorios
    CXPB, MUTPB, NGEN = 0.5, 0.2, 20 # Probabilidades de cruza, muta y numero de generaciones
    pop = toolbox.population() # generación de la población inicial
    hof = tools.HallOfFame(1) # almacena al mejor individuo encontrado en todas las generaciones
    stats = tools.Statistics(lambda ind: ind.fitness.values) # generar las estadísticas de población sobre el fitness
    stats.register("avg", np.mean) #calcular la media
    stats.register("std", np.std)  #desviación estándar típica
    stats.register("min", np.min)  #valor mínimo absoluto de la generación
    stats.register("max", np.max)  #valor máximo absoluto de la generación
    logbook = tools.Logbook()
    pop, logbook = algorithms.eaSimple(pop, toolbox, cxpb=CXPB, mutpb=MUTPB, ngen=NGEN, stats=stats, halloffame=hof, verbose=True)
    return hof, logbook

if __name__ == "__main__":
    best, log = main()
    print("Mejor fitness: %f" %best[0].fitness.values)
    print("Mejor individuo %s" %best[0])
    plot_evolucion(log)
```

11. Convergencia del algoritmo

La función `plot_evolucion()` se debe ejecutar una vez termine el algoritmo genético

```python
"""
Representa la evolución del mejor individuo en cada generación
"""
def plot_evolucion(log):
    gen = np.array(log.select("gen"))
    fit_mins = np.array(log.select("min"), dtype=float)
    fit_maxs = np.array(log.select("max"), dtype=float)
    fit_ave = np.array(log.select("avg"), dtype=float)

    fig, ax1 = plt.subplots()
    ax1.plot(gen, fit_mins, "b")
    ax1.plot(gen, fit_maxs, "r")
    ax1.plot(gen, fit_ave, "--k")
    where_mask = fit_maxs >= fit_mins
    ax1.fill_between(gen, fit_mins, fit_maxs, where=where_mask, facecolor='g', alpha = 0.2)
    ax1.set_xlabel("Generation")
    ax1.set_ylabel("Fitness")
    ax1.set_ylim([-10, 160])
    ax1.legend(["Min", "Max", "Avg"], loc="lower center")
    plt.grid(True)
    plt.savefig("Convergencia.eps", dpi = 300)
```



Se muestra la evolución del algoritmo genético representando tres curvas:

- en azul se muestra el valor mínimo de la función objetivo en cada generación,
- en color negro con línea discontinua se muestra el valor medio en cada generación y, por último,
- en rojo se muestra el valor máximo en cada generación.

Se puede observar que el algoritmo **converge** a partir de la *generación número 10*.

Al ser un problema sumamente sencillo, prácticamente desde la primera generación aparecen soluciones cercanas al mejor individuo.

No obstante, se puede observar que el valor medio de la generación va en aumento durante las primeras generaciones. Esto es así hasta que la mayoría de los individuos de la población son parecidos, por lo que el valor medio de la generación se estabiliza.

Los picos (tanto en el valor mínimo como en el valor medio) se deben a que, a causa de las operaciones genéticas, pueden aparecer individuos que no se ajusten bien a la función objetivo, por lo que generen un fitness muy bajo que afectará tanto a la media como al valor mínimo.

12. Resultados obtenidos

gen	nevals	avg	std	min	max
0	10	81.3622	28.287	31.2954	116.497
1	2	108.322	14.1648	71.337	126.071
2	7	116.868	9.20852	98.8946	136.779
3	4	121.084	8.49836	112.226	136.779
4	7	125.454	8.40708	116.497	136.779
5	4	135.519	3.19937	126.071	136.779
6	5	136.779	0	136.779	136.779
7	5	136.779	0	136.779	136.779
8	10	136.779	0	136.779	136.779
9	4	136.358	1.26178	132.573	136.779
10	6	136.779	0	136.779	136.779
11	6	136.779	0	136.779	136.779
12	7	136.779	0	136.779	136.779
13	7	136.779	0	136.779	136.779
14	8	122.639	41.2271	-1	136.779
15	6	136.779	0	136.779	136.779
16	5	136.779	0	136.779	136.779
17	6	136.779	0	136.779	136.779
18	6	136.779	0	136.779	136.779
19	6	136.452	0.980645	133.51	136.779
20	4	136.779	0	136.779	136.779

Mejor fitness: 136.778896

Mejor individuo [-94.69280606322728, -98.7002480643878]

13. Análisis de los resultados
    - Podemos observar que el valor obtenido está muy cerca del valor óptimo real, pero no es exactamente el mismo.
    - A lo largo de las generaciones, el valor medio **avg** de la población se va incrementando. Esto es positivo ya que significa que los individuos que forman la población son, en media, mejores.
    - La desviación típica **std** va disminuyendo, en general. Esto indica que los individuos cada vez son más parecidos. Esto es esperable ya que el algoritmo genético es elitista por el proceso de selección, por lo que los individuos tenderán a parecerse.
    - El valor máximo **max** va aumentando a lo largo de las generaciones. Esto es tremendamente positivo ya que indica que el algoritmo está funcionando correctamente.

14. Implementar el algoritmo para una población de 30 y 50 individuos, con 20 generaciones, Hacer una tabla comparativa de los resultados obtenidos y las gráficas de convergencia.

Notas:

- Este problema de optimización, se puede abordar con cualquier otro modelo.
- Los algoritmos genéticos se deben implementar en los problemas de optimización, en los que la solución óptima no se conozca, no se tenga una aproximación de ella o incluso no se tenga la certeza de que podamos obtenerla.
- Cuanto mayor sea el número de variables independientes mayor debe ser el tamaño de la población inicial. Por ejemplo, si tenemos un problema con dos variables, podemos empezar con una población de 20 individuos. El tamaño de la población inicial debe garantizar que haya una amplia diversidad de genes en los individuos. Este factor potenciará la fuerza de exploración y explotación del algoritmo.
- En el caso de problemas de maximización, cuanto mayor sea el fitness, mayor será la probabilidad de participar en las operaciones de cruce y mutación.

Referencia: Gutiérrez Reina, D. Tapia Córdoba, A. y Rodríguez del Nozal, Á. (2020). Algoritmos Genéticos con Python: (1 ed.). Barcelona (España), Marcombo. Recuperado de https://elibro.net/es/ereader/mayab/280030